Lecture: Permutations using Recursion / Backtracking

1. Introduction

Permutation means **arranging elements in all possible orders**.
Unlike subsets or combinations, where order doesn't matter — in **permutations**, order is everything.

For example:
For [1,2,3],
✅ [1,2,3] and [2,1,3] are **different permutations**.

2. Problem Statement

Given an array of distinct integers nums, return all possible **permutations** of the elements.

3. Example

Input:

nums = [1, 2, 3]

Output:

[[1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]]

✅ There are 3! = 6 total permutations.

4. Key Concept

In **combinations**, order doesn't matter.
In **permutations**, order matters, so we need to **swap or choose elements differently at each step**.

We use **backtracking** to:

- Fix one element at a time.
- Recursively permute the remaining ones.
- Backtrack (undo choices) to explore other possibilities.

5. Real-Life Analogy

Imagine arranging **3 books** on a shelf:

- You can place any book first,

- Then choose the next from the remaining,
- And finally the last.

That's exactly what permutations do — explore all possible orders.

6. Recursive Intuition

At each recursion level:

1. Pick one element as the current choice.
2. Recursively generate all permutations of the remaining elements.
3. After exploring, undo (backtrack) the choice.

7. Recursive Tree Example

For [1,2,3]:

```
      []
    /   |   \
  [1]   [2]   [3]
  / \   / \   / \
[1,2][1,3][2,1][2,3][3,1][3,2]
```

✅ Output → All 6 permutations

8. Pseudocode — Standard Recursive Method

```
def permute(nums):
  result = []


  def backtrack(current, remaining):
    # Base case: all elements used
    if not remaining:
      result.append(current[:])
```

```
return

    for i in range(len(remaining)):
        # Choose element
        current.append(remaining[i])

        # Recurse with remaining elements except chosen one
        backtrack(current, remaining[:i] + remaining[i+1:])

        # Backtrack (undo choice)
        current.pop()

  backtrack([], nums)
  return result
```

✅ Elegant recursion pattern
✅ Automatically explores all orderings

9. Step-by-Step Example

Input: [1,2,3]

Step	Current	Remaining	Result Added
Start	[]	[1,2,3]	—
Choose 1	[1]	[2,3]	—
Choose 2	[1,2]	[3]	[1,2,3]
Backtrack	[1]	[2,3]	—
Choose 3	[1,3]	[2]	[1,3,2]

Step	Current	Remaining	Result Added
Repeat for 2 & 3 as start	...	...	✅ all permutations

✅ Output = [[1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]]

10. Alternative — In-Place Swapping Method

This method avoids creating new lists at each step, by **swapping elements** in-place.

```
def permute_swap(nums):
  result = []

  def backtrack(start):
    if start == len(nums):
      result.append(nums[:])
      return

    for i in range(start, len(nums)):
      nums[start], nums[i] = nums[i], nums[start]  # swap
      backtrack(start + 1)
      nums[start], nums[i] = nums[i], nums[start]  # backtrack

  backtrack(0)
  return result
```

✅ More memory efficient (no slicing)
✅ Commonly used in interviews

11. Permutations with Duplicates (Unique Permutations)

If the array contains **duplicates**, we need to avoid repeating the same permutation.

```
def permute_unique(nums):
    result = []
    nums.sort()
    used = [False] * len(nums)

    def backtrack(current):
        if len(current) == len(nums):
            result.append(current[:])
            return

        for i in range(len(nums)):
            if used[i]:
                continue
            if i > 0 and nums[i] == nums[i - 1] and not used[i - 1]:
                continue

            used[i] = True
            current.append(nums[i])
            backtrack(current)
            current.pop()
            used[i] = False

    backtrack([])
    return result
```

✅ Handles duplicates by sorting and skipping same consecutive elements.

Example:
nums = [1,1,2] → [[1,1,2],[1,2,1],[2,1,1]]

12. Complexity Analysis

Aspect	Complexity
Total Permutations	n!
Time Complexity	O(n × n!)
Space Complexity	O(n) (recursion + temp list)

13. Iterative Approach (Optional)

```
def permute_iterative(nums):
    result = [[]]
    for num in nums:
        new_result = []
        for perm in result:
            for i in range(len(perm) + 1):
                new_result.append(perm[:i] + [num] + perm[i:])
        result = new_result
    return result
```

✅ Builds permutations by inserting elements in every position.

14. Edge Cases

Case	Output
Empty list []	[[]]
Single element [5]	[[5]]
Duplicate elements [1,1]	Handled with permute_unique()
Large list	Quickly grows factorially — use pruning

15. Real-World Applications

Domain	Usage
Cryptography	Key / password generation
Scheduling	Generate all possible task orders
Testing	Input reordering for test cases
AI & Games	Path order exploration
Mathematics	Factorial-based counting problems

16. Practice Tasks

1. Implement **standard recursive permutation**.
2. Implement **in-place swap version**.
3. Implement **unique permutations** (handle duplicates).
4. Generate **permutations of size k** (optional).
5. Compare output with Python's built-in itertools.permutations().

17. Summary Table

Feature	Permutations
Order Matters	✅ Yes
Duplicates Allowed	❌ (unless handled)
Recursion Type	Backtracking
Total Results	n!
Time Complexity	O(n × n!)
Common Uses	Arrangements, order-based problems

18. Closing Thought

The **Permutations problem** completes the trilogy of **Subsets → Combinations → Permutations**,
teaching you:

- How recursion explores **different decision spaces**,
- How backtracking reverts choices,
- And how order transforms the output space.